

CSE 3204
Formal Language, Automata and Computability



Lecture 17
CNF and Pumping Lemma for CFL

Construction of Chomsky Normal Form (CNF)

- A safe order is
 - Eliminate ϵ -productions
 - Eliminate unit production
 - Eliminate useless symbols

Theorem 7.14: If G is a CFG generating a language that contains at least one string other than ϵ , then there is another CFG G_1 such that $L(G_1) = L(G) - \{\epsilon\}$, and G_1 has no ϵ -productions, unit productions, or useless symbols.

We complete our study of grammatical simplifications by showing that every nonempty CFL without ϵ has a grammar G in which all productions are in one of two simple forms, either:

1. $A \rightarrow BC$, where A , B , and C , are each variables, or
2. $A \rightarrow a$, where A is a variable and a is a terminal.

Further, G has no useless symbols. Such a grammar is said to be in *Chomsky Normal Form*, or CNF.¹

To put a grammar in CNF, start with one that satisfies the restrictions of Theorem 7.14; that is, the grammar has no ϵ -productions, unit productions, or useless symbols. Every production of such a grammar is either of the form $A \rightarrow a$, which is already in a form allowed by CNF, or it has a body of length 2 or more. Our tasks are to:

- a) Arrange that all bodies of length 2 or more consist only of variables.
- b) Break bodies of length 3 or more into a cascade of productions, each with a body consisting of two variables.

Construction of CNF

The construction for (a) is as follows. For every terminal a that appears in a body of length 2 or more, create a new variable, say A . This variable has only one production, $A \rightarrow a$. Now, we use A in place of a everywhere a appears in a body of length 2 or more. At this point, every production has a body that is either a single terminal or at least two variables and no terminals.

For step (b), we must break those productions $A \rightarrow B_1 B_2 \cdots B_k$, for $k \geq 3$, into a group of productions with two variables in each body. We introduce $k - 2$ new variables, $C_1, C_2, \dots, C_{k-2}$. The original production is replaced by the $k - 1$ productions

$$A \rightarrow B_1 C_1, \quad C_1 \rightarrow B_2 C_2, \dots, C_{k-3} \rightarrow B_{k-2} C_{k-2}, \quad C_{k-2} \rightarrow B_{k-1} B_k$$

$$\begin{aligned} E &\rightarrow E + T \mid T * F \mid (E) \mid a \mid b \mid Ia \mid Ib \mid I0 \mid I1 \\ T &\rightarrow T * F \mid (E) \mid a \mid b \mid Ia \mid Ib \mid I0 \mid I1 \\ F &\rightarrow (E) \mid a \mid b \mid Ia \mid Ib \mid I0 \mid I1 \\ I &\rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1 \end{aligned}$$

$$\begin{array}{l} \text{Part (a):} \quad A \rightarrow a \quad B \rightarrow b \quad Z \rightarrow 0 \quad O \rightarrow 1 \\ \quad \quad \quad P \rightarrow + \quad M \rightarrow * \quad L \rightarrow (\quad R \rightarrow) \end{array}$$

Part (b):

$$\begin{array}{l} E \rightarrow EPT \mid TMF \mid LER \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\ T \rightarrow TMF \mid LER \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\ F \rightarrow LER \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\ I \rightarrow a \mid b \mid IA \mid IB \mid IZ \mid IO \\ A \rightarrow a \\ B \rightarrow b \\ Z \rightarrow 0 \\ O \rightarrow 1 \\ P \rightarrow + \\ M \rightarrow * \\ L \rightarrow (\\ R \rightarrow) \end{array}$$

Construction of CNF

$$\begin{aligned}
 E &\rightarrow EC_1 \mid TC_2 \mid LC_3 \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\
 T &\rightarrow TC_2 \mid LC_3 \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\
 F &\rightarrow LC_3 \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\
 I &\rightarrow a \mid b \mid IA \mid IB \mid IZ \mid IO \\
 A &\rightarrow a \\
 B &\rightarrow b \\
 Z &\rightarrow 0 \\
 O &\rightarrow 1 \\
 P &\rightarrow + \\
 M &\rightarrow * \\
 L &\rightarrow (\\
 R &\rightarrow) \\
 C_1 &\rightarrow PT \\
 C_2 &\rightarrow MF \\
 C_3 &\rightarrow ER
 \end{aligned}$$

Theorem 7.16: If G is a CFG whose language contains at least one string other than ϵ , then there is a grammar G_1 in Chomsky Normal Form, such that $L(G_1) = L(G) - \{\epsilon\}$.

Greibach Normal Form (GNF):

prove. Every nonempty language without ϵ is $L(G)$ for some grammar G each of whose productions is of the form $A \rightarrow a\alpha$, where a is a terminal and α is a string of zero or more variables. Converting a grammar to

The Size of a Parse Tree

Our first step in deriving a pumping lemma for CFL's is to examine the shape and size of parse trees. One of the uses of CNF is to turn parse trees into binary trees. These trees have some convenient properties, one of which we exploit here.

Theorem 7.17: Suppose we have a parse tree according to a Chomsky-Normal-Form grammar $G = (V, T, P, S)$, and suppose that the yield of the tree is a terminal string w . If the length of the longest path is n , then $|w| \leq 2^{n-1}$.

PROOF: The proof is a simple induction on n .

BASIS: $n = 1$. Recall that the length of a path in a tree is the number of edges, i.e., one less than the number of nodes. Thus, a tree with a maximum path length of 1 consists of only a root and one leaf labeled by a terminal. String w is this terminal, so $|w| = 1$. Since $2^{n-1} = 2^0 = 1$ in this case, we have proved the basis.

INDUCTION: Suppose the longest path has length n , and $n > 1$. The root of the tree uses a production, which must be of the form $A \rightarrow BC$, since $n > 1$; i.e., we could not start the tree using a production with a terminal. No path in the subtrees rooted at B and C can have length greater than $n - 1$, since these paths exclude the edge from the root to its child labeled B or C . Thus, by the inductive hypothesis, these two subtrees each have yields of length at most 2^{n-2} . The yield of the entire tree is the concatenation of these two yields, and therefore has length at most $2^{n-2} + 2^{n-2} = 2^{n-1}$. Thus, the inductive step is proved. $\square$

Statement of the Pumping Lemma

Theorem 7.18: (The pumping lemma for context-free languages) Let L be a CFL. Then there exists a constant n such that if z is any string in L such that $|z|$ is at least n , then we can write $z = uvwxy$, subject to the following conditions:

1. $|vwx| \leq n$. That is, the middle portion is not too long.
2. $vx \neq \epsilon$. Since v and x are the pieces to be “pumped,” this condition says that at least one of the strings we pump must not be empty.
3. For all $i \geq 0$, uv^iwx^iy is in L . That is, the two strings v and x may be “pumped” any number of times, including 0, and the resulting string will still be a member of L .

Figure 7.5 suggests the longest path in the tree for z , where k is at least m and the path is of length $k+1$. Since $k \geq m$, there are at least $m+1$ occurrences of variables $A_0, A_1, \dots, A_k$ on the path. As there are only m different variables in V , at least two of the last $m+1$ variables on the path (that is, A_{k-m}

PROOF: Our first step is to find a Chomsky-Normal-Form grammar G for L . Technically, we cannot find such a grammar if L is the CFL $\emptyset$ or $\{\epsilon\}$. However, if $L = \emptyset$ then the statement of the theorem, which talks about a string z in L surely cannot be violated, since there is no such z in $\emptyset$. Also, the CNF grammar G will actually generate $L - \{\epsilon\}$, but that is again not of importance, since we shall surely pick $n > 0$, in which case z cannot be ϵ anyway.

Now, starting with a CNF grammar $G = (V, T, P, S)$ such that $L(G) = L - \{\epsilon\}$, let G have m variables. Choose $n = 2^m$. Next, suppose that z in L is of length at least n . By Theorem 7.17, any parse tree whose longest path is of length m or less must have a yield of length $2^{m-1} = n/2$ or less. Such a parse tree cannot have yield z , because z is too long. Thus, any parse tree with yield z has a path of length at least $m+1$.

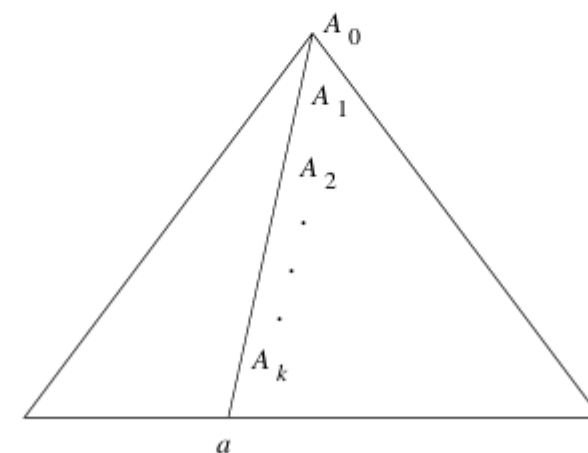


Figure 7.5: Every sufficiently long string in L must have a long path in its parse tree

Statement of the Pumping Lemma

through A_k , inclusive) must be the same variable. Suppose $A_i = A_j$, where $k - m \leq i < j \leq k$.

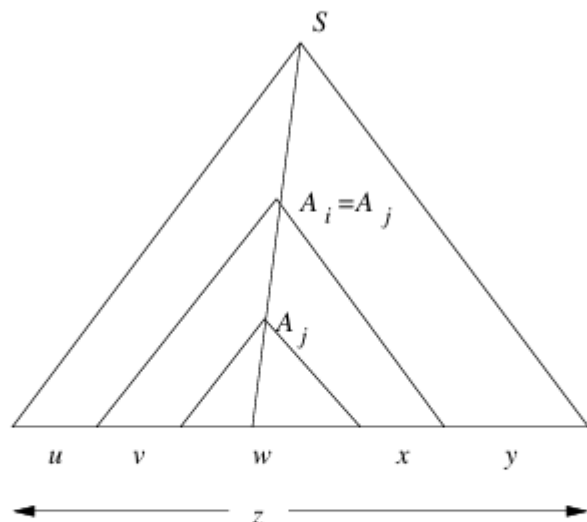


Figure 7.6: Dividing the string w so it can be pumped

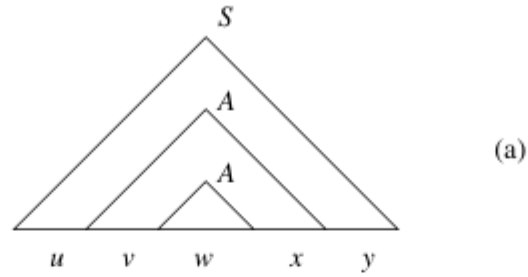
Then it is possible to divide the tree as shown in Fig. 7.6. String w is the yield of the subtree rooted at A_j . Strings v and x are the strings to the left and right, respectively, of w in the yield of the larger subtree rooted at A_i . Note that, since there are no unit productions, v and x could not both be ϵ , although one could be. Finally, u and y are those portions of z that are to the left and right, respectively, of the subtree rooted at A_i .

If $A_i = A_j = A$, then we can construct new parse trees from the original tree, as suggested in Fig. 7.7(a). First, we may replace the subtree rooted at A_i , which has yield $vw x$, by the subtree rooted at A_j , which has yield w . The reason we can do so is that both of these trees have root labeled A . The resulting tree is suggested in Fig. 7.7(b); it has yield $uw y$ and corresponds to the case $i = 0$ in the pattern of strings $uv^i w x^i y$.

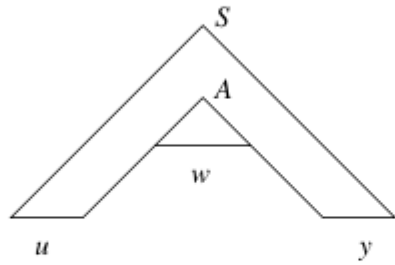
Another option is suggested by Fig. 7.7(c). There, we have replaced the subtree rooted at A_j by the entire subtree rooted at A_i . Again, the justification is that we are substituting one tree with root labeled A for another tree with the same root label. The yield of this tree is $uv^2 w x^2 y$. Were we to then replace the subtree of Fig. 7.7(c) with yield w by the larger subtree with yield $vw x$, we would have a tree with yield $uv^3 w x^3 y$, and so on, for any exponent i . Thus, there are parse trees in G for all strings of the form $uv^i w x^i y$, and we have almost proved the pumping lemma.

The remaining detail is condition (1), which says that $|vw x| \leq n$. However, we picked A_i to be close to the bottom of the tree; that is, $k - i \leq m$. Thus, the longest path in the subtree rooted at A_i is no greater than $m + 1$. By Theorem 7.17, the subtree rooted at A_i has a yield whose length is no greater than $2^m = n$. $\square$

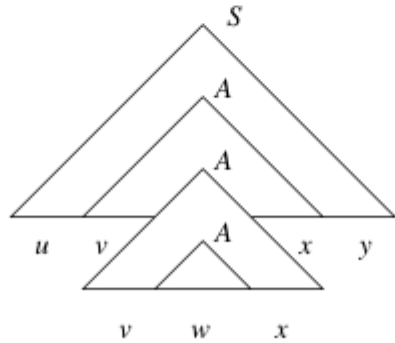
Statement of the Pumping Lemma



(a)



(b)



(c)

Figure 7.7: Pumping strings v and x zero times and pumping them twice

Application of Pumping Lemma for CFL

Example 7.19 : Let L be the language $\{0^n 1^n 2^n \mid n \geq 1\}$. That is, L consists of all strings in $0^+ 1^+ 2^+$ with an equal number of each symbol, e.g., 012, 001122, and so on. Suppose L were context-free. Then there is an integer n given to us by the pumping lemma.² Let us pick $z = 0^n 1^n 2^n$.

Suppose the “adversary” breaks z as $z = uvwxy$, where $|vwx| \leq n$ and v and x are not both ϵ . Then we know that vwx cannot involve both 0’s and 2’s, since the last 0 and the first 2 are separated by $n + 1$ positions. We shall prove that L contains some string known not to be in L , thus contradicting the assumption that L is a CFL. The cases are as follows:

1. vwx has no 2’s. Then vx consists of only 0’s and 1’s, and has at least one of these symbols. Then uvw , which would have to be in L by the pumping lemma, has n 2’s, but has fewer than n 0’s or fewer than n 1’s, or both. It therefore does not belong in L , and we conclude L is not a CFL in this case.
2. vwx has no 0’s. Similarly, uvw has n 0’s, but fewer 1’s or fewer 2’s. It therefore is not in L .

Whichever case holds, we conclude that L has a string we know not to be in L . This contradiction allows us to conclude that our assumption was wrong; L is not a CFL. $\square$

CFL can match two groups of symbols for equality or non-equality, they cannot match three such groups.

Example 7.20 : Let L be the language $\{0^i 1^j 2^i 3^j \mid i \geq 1 \text{ and } j \geq 1\}$. If L is context-free, let n be the constant for L , and pick $z = 0^n 1^n 2^n 3^n$. We may write $z = uvwxy$ subject to the usual constraints $|vwx| \leq n$ and $vx \neq \epsilon$. Then vwx is either contained in the substring of one symbol, or it straddles two adjacent symbols.

If vwx consists of only one symbol, then uvw has n of three different symbols and fewer than n of the fourth symbol. Thus, it cannot be in L . If vwx straddles two symbols, say the 1’s and 2’s, then uvw is missing either some 1’s or some 2’s, or both. Suppose it is missing 1’s. As there are n 3’s, this string cannot be in L . Similarly, if it is missing 2’s, then as it has n 0’s, uvw cannot be in L . We have contradicted the assumption that L is a CFL and conclude that it is not. $\square$

CFLs cannot do is match two pairs of equal numbers of symbols provided that the pairs interleavetime

Application of Pumping Lemma for CFL

Example 7.21 : Let $L = \{ww \mid w \text{ is in } \{0,1\}^*\}$. That is, L consists of repeating strings, such as ϵ , 0101, 00100010, or 110110. If L is context-free, then let n be its pumping-lemma constant. Consider the string $z = 0^n 1^n 0^n 1^n$. This string is $0^n 1^n$ repeated, so z is in L .

Following the pattern of the previous examples, we can break $z = uvwx$, such that $|vwx| \leq n$ and $vx \neq \epsilon$. We shall show that uwy is not in L , and thus show L not to be a context-free language, by contradiction.

First, observe that, since $|vwx| \leq n$, $|uwy| \geq 3n$. Thus, if uwy is some repeating string, say tt , then t is of length at least $3n/2$. There are several cases to consider, depending where vwx is within z .

1. Suppose vwx is within the first n 0's. In particular, let vx consist of k 0's, where $k > 0$. Then uwy begins with $0^{n-k}1^n$. Since $|uwy| = 4n - k$, we know that if $uwy = tt$, then $|t| = 2n - k/2$. Thus, t does not end until after the first block of 1's; i.e., t ends in 0. But uwy ends in 1, and so it cannot equal tt .
2. Suppose vwx straddles the first block of 0's and the first block of 1's. It may be that vx consists only of 0's, if $x = \epsilon$. Then, the argument that uwy is not of the form tt is the same as case (1). If vx has at least one 1, then we note that t , which is of length at least $3n/2$, must end in 1^n , because uwy ends in 1^n . However, there is no block of n 1's except the final block, so t cannot repeat in uwy .

3. If vwx is contained in the first block of 1's, then the argument that uwy is not in L is like the second part of case (2).
4. Suppose vwx straddles the first block of 1's and the second block of 0's. If vx actually has no 0's, then the argument is the same as if vwx were contained in the first block of 1's. If vx has at least one 0, then uwy starts with a block of n 0's, and so does t if $uwy = tt$. However, there is no other block of n 0's in uwy for the second copy of t . We conclude in this case too, that uwy is not in L .
5. In the other cases, where vwx is in the second half of z , the argument is symmetric to the cases where vwx is contained in the first half of z .

Thus, in no case is uwy in L , and we conclude that L is not context-free. $\square$

- Section 7.1 and 7.2 of Chapter 7